Heterogeneous Active Retrieval Across Structurally Different Evidence Substrates

Working Draft

August 2026

Abstract

Adaptive generation-time retrieval is established prior art. We test a narrower systems question: do structurally different information needs benefit from distinct read-only evidence coprocessors rather than one universal text retriever? We implement typed relational, lexical, semantic-vector, and freshness-aware controlled-web adapters behind a credential-isolating registry. On a 22-item source-optimal controlled benchmark, source-native oracle and transparent heuristic routing both reach 22/22, including safe abstention on a conflicting fresh source. A matched universal textual index reaches 13/22 and supports 7/16 retrieval-required answers; it loses relational aggregate/join and source-status semantics. Broadcast also reaches 22/22 but averages 2.91 calls per example versus 0.73 for routed retrieval. Explicit descriptor burden grows from 4.0 to 22.5 mean tokens as the available catalog grows from one to four sources, while the runtime registry adds no recurring descriptor tokens. Replacing the controlled relational, lexical, and vector implementations with DuckDB, SQLite FTS5, and exact-flat FAISS preserves all 154 matched four-source decisions and complete required provenance for all three substituted sources. These remain small controlled runtime results. A second 11-case enterprise-style fixture uses real local Iceberg snapshots, governed metrics, and an Oxigraph ontology: native composition reaches 11/11 while a top-5 textualized baseline reaches 2/11. This is still deterministic local evidence, not language-model or production-service evidence. A pinned 320-question TAT-QA diagnostic then separates retrieval from compute: 129/320 questions require computation, all 106 sampled gold arithmetic derivations execute exactly, but the current runtime has no TAT-QA table/text operation adapter. At matched top-5 retrieval, adding table headers to a hybrid lexical ranker raises evidence recall from 0.751 to 0.799 over 308 evaluable questions. This is a retrieval/composition audit, not end-to-end question answering. The pre-registered learned-router gate is NO-GO: native heterogeneity has value, context burden grows, and broadcast costs more, but the heuristic leaves no routing gap.

1 Question and Claim Boundary

FLARE and Self-RAG establish important forms of adaptive retrieval [1, 2]. Our question is whether source-native semantics matter after retrieval need has been identified. The core paper therefore keeps need and source routing transparent. It does not claim a learned router, production web-search quality, or neural answer use.

The factorized pipeline is

$$\text{NEED} \rightarrow \text{SOURCE} \rightarrow \text{QUERY} \rightarrow \text{RETRIEVE} \rightarrow \text{EVIDENCE STATUS} \rightarrow \text{USE}.$$

Paper 1.5 can provide a typed retrieval-required event. Paper 2.5 studies the source and query stages without importing a learned multi-source policy.

2 Read-Only Source Runtime

Every request contains a request identifier, source type, operation, typed payload, and bounds. Every evidence record contains source type and version, record identifier, value/content, observation time, relevance, support status, provenance, latency, and retrieved bytes. The runtime accepts only sources whose side-effect class is read-only. Credential scopes remain in the registry and are removed from public catalog descriptors.

2.1 Source-native adapters

- **Relational DB:** an embedded SQLite database with parameterized, typed templates for lookup, sum, count, average, argmax, and join.
- **Lexical index:** exact document/phrase retrieval over versioned policy and handbook records.
- **Vector index:** deterministic semantic-vector matching over reports, including controlled synonym normalization.
- **Fresh web analogue:** dated public-feed records with one explicit two-record conflict. It exercises freshness and conflict policy without network variability.

These are genuinely different access semantics. Multiple wrappers over one text index would not satisfy the design.

The backend interface is independently swappable. A local production-shaped suite substitutes DuckDB parameterized SQL, SQLite FTS5 ranking, and FAISS `IndexFlatIP` retrieval while retaining the same typed requests and evidence records. Each substituted record adds backend/version, resource, normalized query, record IDs, and snapshot to provenance. The web source remains the controlled analogue in this pass.

3 Routing and Typed Queries

The heuristic routes structured attributes and aggregates to DB, exact documents/clauses to lexical retrieval, paraphrased report questions to vector retrieval, and current/public requests to the fresh source. After source selection, source-specific fields compile to bounded requests such as `db.max_sales(year=2026)` or `web.lookup(entity,relation,time_window)`. Generated SQL and arbitrary code execution are forbidden.

4 Benchmark and Conditions

The frozen benchmark contains six DB tasks, three lexical tasks, three vector tasks, four web tasks, and six supplied-context controls. DB tasks cover every implemented operation. The web set contains a conflict whose authorized answer is abstention. Nested catalogs expose 1, 2, 3, and 4 source types.

For every catalog we run seven conditions:

1. oracle need, source, and query;
2. real typed need with oracle source;
3. real typed need with heuristic source;

Table 1: Four-source catalog over 22 examples. Support and UCR use the 16 retrieval-required tasks. Calls and fanout average over all examples.

Condition	Final	Support	UCR	Calls	Descriptor tok.
Native oracle	1.000	1.000	0.000	0.73	0.0
Heuristic routing	1.000	1.000	0.000	0.73	0.0
Explicit schemas	1.000	1.000	0.000	0.73	22.5
Universal text	0.591	0.438	0.562	0.73	3.6
Broadcast	1.000	1.000	0.000	2.91	0.0

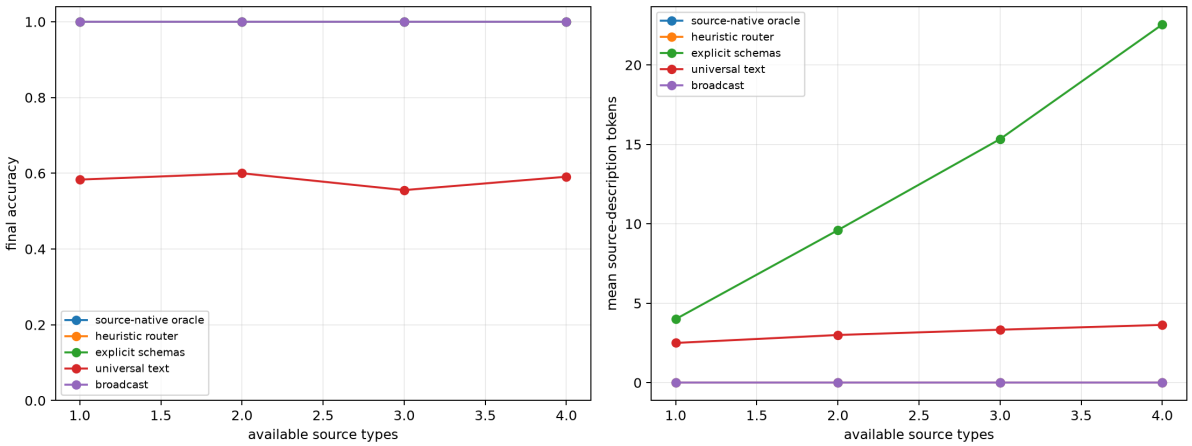

Figure 1: Final accuracy and recurring source-description tokens as source count grows. Registry capabilities remain outside neural context; explicit schemas grow with the catalog.

4. heuristic need and source;
5. explicit source selection with all descriptors;
6. one universal textual retriever;
7. broadcast to every available native source.

This yields 469 predictions and matching factorized traces. The universal index contains textual snapshots of every available source record and returns the top lexical match. It is intentionally strong on direct textual facts, but it cannot execute relational aggregates or preserve native conflict semantics.

5 Controlled Results

Native execution and routing are perfect by construction on this small source-optimal diagnostic, so the result is not a claim of broad routing accuracy. The useful comparison is semantic headroom: universal textualization misses nine examples that native source operations resolve. Broadcasting removes routing risk but performs four calls per retrieval-required example. The routed conditions call exactly one source on each of 16 needs and none on six controls.

Table 2: Local backend substitution on oracle-source rows. Latency is mean source execution time on one Windows host, not a backend ranking.

Backend	Rows	Final	Provenance	Mean ms
DuckDB	6	1.000	1.000	0.742
SQLite FTS5	3	1.000	1.000	0.036
FAISS flat	3	1.000	1.000	0.047

5.1 Conflict and composition

The fresh-source conflict returns two provenance-bearing CONFLICT records. Native conditions accept abstention rather than selecting a value. Twenty-four additional deterministic compositions test entity resolver → DB and date resolver → web. Both stages and final outputs are 24/24, and every request records a dependency DAG. These are plumbing/economics checks, not model planning evidence.

5.2 Local production-backend substitution

We rerun the complete four-source matrix after substituting DuckDB 1.5.5 for the embedded relational implementation, SQLite 3.49.1 FTS5 for lexical scanning, and FAISS 1.15.0 exact-flat inner-product search for the controlled vector search. All 154 matched predictions retain the same final and evidence-support decisions. Table 2 isolates oracle-source rows for the three replaced backends. Required provenance completeness checks backend/version, resource, normalized query, record IDs, and snapshot.

This validates adapter substitution and provenance retention on the frozen data; it does not test service deployment, persistence, concurrency, failure recovery, or realistic corpus scale. No Docker service was started. Postgres/pgvector, Iceberg REST catalogs, Qdrant, and Weaviate are outside this completed local phase.

For the next boundary, the repository includes a WSL2-only Compose definition, seed SQL, health check, explicit Postgres and pgvector adapters, and integration tests gated by a caller-supplied DSN. Both adapters use parameter binding and read-only transactions; unavailable services skip rather than fall back. These assets were statically checked only. They contribute no result or latency claim. The same boundary now includes a pinned, localhost-bound Qdrant profile and a retrieval-only adapter, plus an explicit-endpoint PyIceberg REST-catalog adapter. Unit tests use injected clients/catalogs; live tests require caller-supplied URLs and otherwise skip. Neither service path was executed for this paper.

5.3 Iceberg, governed metrics, and ontology composition

We next create four local PyIceberg tables—sales, products, inventory, and customers—through a SQLite development catalog. Sales has two committed snapshots: the first totals 500.0 revenue; after adding an optional `channel` field and appending three rows, the current snapshot totals 1050.0. DuckDB reads the explicit Iceberg metadata directly. The catalog creates the fixture but is not the query engine.

A versioned semantic layer compiles `gross_margin` and `net_revenue` requests to bounded, parameterized DuckDB/Iceberg SQL. An in-process Oxigraph store resolves product-family membership through SPARQL. The mixed path is

ontology concept → product IDs → metric definition → Iceberg snapshot → typed evidence.

Table 3: Enterprise-style local fixture. The universal condition retrieves the five highest-overlap textualized rows, metric descriptions, ontology triples, and policies; it performs no hidden native aggregate.

Condition	Correct	Total	Mean calls
Native governed composition	11	11	1.09
Universal text top-5	2	11	1.00

For example, `hardware-products` resolves to Aster and Cedar, whose governed 2026 gross margin is 39.20%. Provenance retains backend/version, normalized query, record IDs, snapshot ID, metric definition/version, and ontology concept/relation IDs.

The native condition is exact on two direct lookups, one cross-table join, one aggregate, one historical snapshot, one governed metric, two ontology queries, one ontology–metric composition, one document query, and one mixed document–metric query. Universal text supports only the exact policy and one single-member ontology answer. It does not reconstruct the aggregate, join, snapshot, governed metric, multi-member group, or mixed answers. This is a source-semantics diagnostic, not a scale benchmark: the fixture has six sales rows and the query/compiler paths are deterministic.

5.4 Four generic tools: governed result transport

The shared model-facing baseline exposes exactly `__compute`, `__retrieve`, `__verify`, and `__help`; backend descriptors remain in R2. We transport the 11 native-governed enterprise outcomes through a generic `__help` result message and compare them with the typed CogCop records. Answers, provenance, and source-call fields agree on 11/11, and both retain deterministic final accuracy 1.000. The four schemas occupy 72 lexical tokens at registry stages containing 3, 6, and 9 capabilities.

This is an oracle-timed result-transport invariant. It reuses the precomputed shared native-governed R2 run; no model chooses intent, source, query, or timing. Consequently it does not measure voluntary retrieval recall, malformed arguments, extra turns, evidence timing, CONTINUE, or Automatic Rescue Rate. It prevents later comparisons from attributing a backend or provenance advantage to CogCop merely because the generic-tool condition was weakened.

6 Public TAT-QA Composition Audit

TAT-QA combines financial tables and text with extraction and arithmetic questions [3]. We pin the official development file at repository revision `c96247f5`, verify its SHA-256 digest, and freeze 320 of its 1,644 questions by deterministic stratified selection over answer type, evidence source, scale, and comparison requirement. Retained artifacts redistribute only question identifiers, source coordinates, strata, and content hashes; the licensed source remains in a local cache.

This first public-data checkpoint factorizes the requested path as

$$\text{RETRIEVE} \rightarrow \text{EXTRACT} \rightarrow \text{COMPUTE} \rightarrow \text{ANSWER}.$$

The audit marks table, text, and joint table–text requirements from dataset metadata and safely evaluates only gold arithmetic derivations composed of decimal literals, percentages, parentheses, and the four basic operators. It does not expose gold table cells or derivations to a model.

Table 4: Pinned TAT-QA development diagnostic. Compute exactness is conditional on the 106 executable arithmetic derivations, not an end-to-end QA score.

Audit quantity	Count	Rate
Compute required (arithmetic or count)	129/320	0.403
Joint table–text evidence	130/320	0.406
Executable gold arithmetic	106/320	0.331
Exact executable arithmetic	106/106	1.000
End-to-end typed operation adapter	0/320	0.000

Table 5: Matched top-5 retrieval on 308/320 questions with defensible evidence labels. Gold annotations are used for scoring only.

Condition	Mean evidence recall	Complete evidence
Flattened word BM25	0.704	0.601
Flattened character BM25	0.756	0.646
Flattened hybrid	0.751	0.643
Header-aware structured hybrid	**0.799**	**0.695**

Here adapter coverage means a typed end-to-end operation adapter; it remains zero. We separately test whether preserving table structure helps retrieve the evidence that such an adapter would consume. Word and character BM25 rank a matched pool of flattened table rows and paragraphs. Their reciprocal-rank hybrid is compared with the same hybrid over rows enriched with row and column headers. All conditions receive only the question and return five chunks. Ranking never receives answers, derivations, or relevance labels.

The paired structured-minus-flat recall difference is 0.048 (10,000-sample bootstrap 95% interval [0.020, 0.077]); complete retrieval has 24 wins, 8 losses, and 276 ties. The gain appears on table-only (0.665 to 0.747 recall) and joint table–text (0.764 to 0.793) questions, while text-only recall is unchanged at 0.977. This pattern supports retaining table headers, but it does not establish source-native execution: table relevance for arithmetic and spans is derived post hoc from gold operands/answers, 12 questions lack defensible labels, and operation choice remains oracle. LLM-only, generic-tool, typed operation, vector/hybrid semantic, and final-answer conditions remain pending. This diagnostic does not reopen the Paper 3.5 routing gate.

7 Paper 3.5 Gate

Learned source routing requires all four pre-registered criteria:

1. source-native oracle value over universal retrieval: **pass**;
2. a measurable heuristic-to-oracle gap: **fail**;
3. source descriptor burden grows with catalog size: **pass**;
4. broadcast does not dominate quality and cost: **pass**.

The decision is therefore NO-GO. The negative gate is substantive: a learned router would add training and failure surface without measured routing headroom on this benchmark. A later, independently frozen natural-language set may reopen the gate; this result does not. The enterprise

fixture adds native semantic value but does not measure a new source-routing gap, so it does not reverse the gate.

8 Limitations and Falsification

The corpus is synthetic, tiny, single-seed, and designed so source semantics are clear. The heuristic sees controlled lexical cues and therefore should not be generalized to natural requests. FAISS indexes the same controlled semantic features rather than a production embedding model. The web adapter is local and cannot measure network failures, live ranking, or monetary cost. Final accuracy is deterministic evidence acceptance, not LLM evidence use. Backend substitution does not establish service reliability or enterprise-data validity, and all latencies are local prototype measurements.

The enterprise fixture is deliberately tiny and locally generated. Its absolute Iceberg file locations are regenerated by the CLI for each workspace. The semantic definitions and ontology are authored fixtures rather than imported business governance. REST-catalog behavior, concurrent commits, failure recovery, remote object storage, model-authored SQL, and natural-language metric parsing are not tested. Accordingly, 11/11 establishes bounded composition and provenance, not production accuracy.

The public TAT-QA sample is one deterministic development split and uses dataset metadata plus gold derivations only to audit composition availability. It does now measures lexical evidence selection, but table labels for spans and arithmetic are induced from gold answers/operands and 12 rows are not evaluable. It does not measure value extraction, operation prediction, model answer use, latency, semantic-vector retrieval, or robustness. Header-aware lexical recall is not equivalent to a source-native execution advantage. Likewise, the four-tool enterprise audit demonstrates only equal transport of already computed governed records. Model-facing intent and R2 invocation remain unmeasured.

Heterogeneous routing would be weakened if a stronger matched universal system executes aggregates, preserves status/provenance, and closes the 9/22 gap at comparable cost; if broadcast is cheap enough to dominate; or if natural source-selection errors erase native source value.

9 Reproduction

Code is under `src/ccpu/paper2_5`; reusable evidence contracts are under `src/ccpu/common`. From the repository root:

```
python -m ccpu paper2.5 freeze \
  --output-dir artifacts/paper2_5/next_iter/data_v2
python -m ccpu paper2.5 run \
  --benchmark artifacts/paper2_5/next_iter/data_v2/benchmark.jsonl \
  --source-count 4 \
  --output-dir artifacts/paper2_5/next_iter/source_n4_v2
python -m ccpu paper2.5 analyze --predictions <n1> <n2> <n3> <n4> \
  --output-dir artifacts/paper2_5/next_iter/analysis_v2
python -m ccpu paper2.5 compositions --count-per-family 12 \
  --output-dir artifacts/paper2_5/next_iter/compositions_v1
python -m ccpu paper2.5 run \
  --benchmark artifacts/paper2_5/production_v1/data/benchmark.jsonl \
```

```
  --source-count 4 --backend-suite local_production \
  --output-dir artifacts/paper2_5/production_v1/source_n4
python -m ccpu paper2.5 analyze-production \
  --controlled-predictions <controlled-n4-predictions> \
  --production-predictions <production-n4-predictions> \
  --production-traces <production-n4-traces> \
  --output-dir artifacts/paper2_5/production_v1/substitution
python -m ccpu paper2.5 prepare-enterprise \
  --output-dir artifacts/paper2_5/enterprise_repro/fixture
python -m ccpu paper2.5 run-enterprise \
  --fixture-root artifacts/paper2_5/enterprise_repro/fixture \
  --output-dir artifacts/paper2_5/enterprise_repro/evaluation
python -m ccpu paper2.5 compare-generic-tools \
  --fixture-root artifacts/paper2_5/enterprise_v1/fixture \
  --output-dir artifacts/paper2_5/generic_tools_v1/enterprise_transport
python -m ccpu paper2.5 freeze-public-tatqa \
  --config configs/paper2_5/public_tatqa.json \
  --cache-root <verified-public-cache> \
  --output-dir artifacts/paper2_5/public_benchmarks/tatqa_v1
python -m ccpu paper2.5 analyze-public-tatqa \
  --config configs/paper2_5/public_tatqa.json \
  --cache-root <verified-public-cache> \
  --selection artifacts/paper2_5/public_benchmarks/tatqa_v1/selection.jsonl \
  --output-dir artifacts/paper2_5/public_benchmarks/tatqa_v1
python -m ccpu paper2.5 analyze-public-tatqa-retrieval \
  --config configs/paper2_5/public_tatqa.json \
  --cache-root <verified-public-cache> \
  --selection artifacts/paper2_5/public_benchmarks/tatqa_v1/selection.jsonl \
  --top-k 5 \
  --output-dir artifacts/paper2_5/public_benchmarks/tatqa_retrieval_v1
```

Run manifests hash benchmark, predictions, traces, and summaries and record the repository and environment state. Install the `data` project extra for DuckDB and FAISS. This matrix uses no network, model-token, or container service.

10 Conclusion

Source-native execution has clear controlled value over universal textualization, and routing avoids broadcast fanout while keeping capability descriptors outside recurring context. That result justifies heterogeneous runtime adapters, not a learned router. Because the transparent heuristic closes the oracle routing gap on the present freeze, Paper 3.5 remains gated off. The local substitution result strengthens the interface claim: source-native semantics and provenance survive three real in-process backend replacements. It does not yet establish the broader production-data-stack claim. The tokenizer-aware trigger extension in Papers 1.5 and 2 concerns retrieval need and engine-family routing; it supplies no new source-choice evidence and therefore does not change this paper's routing gate. The local Iceberg/semantic/ontology result then demonstrates one bounded governed composition with snapshot and definition provenance. Its universal gap supports source-native semantics, but the absence of measured routing headroom leaves the learned-router decision

unchanged. The pinned TAT-QA audit adds external task structure and shows that arithmetic is not the current bottleneck. Header-aware lexical retrieval improves matched evidence recall, but does not establish comparative QA performance; typed operation and model conditions define the next Paper 2.5 step. The fixed four-tool gateway also preserves all 11 governed result records with constant visible schema size. A novel invocation claim still requires matched model-facing evidence of automatic rescue, earlier safe retrieval, CONTINUE, or cost beyond schema/PRA savings.

References

[1] Z. Jiang et al. Active retrieval augmented generation. *EMNLP*, 2023. `https://arxiv.org/abs/2305.06983`.

[2] A. Asai et al. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. *ICLR*, 2024. `https://arxiv.org/abs/2310.11511`.

[3] F. Zhu et al. TAT-QA: A question answering benchmark on a hybrid of tabular and textual content in finance. *ACL-IJCNLP*, 2021. `https://arxiv.org/abs/2105.07624`.